



AI-Powered Drone

Group Members: Enzo Emami, Mei Kusuyama, Nancy Ta, Aayush Sugali, Isaac Zhi Kang

ENGR 77

De Anza College

Professor Saied Rafati

Spring 2025

Content Overview



1. What Our Project Does



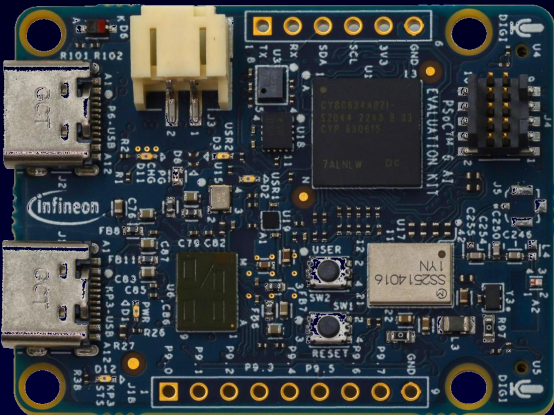
2. Why Is There A Need



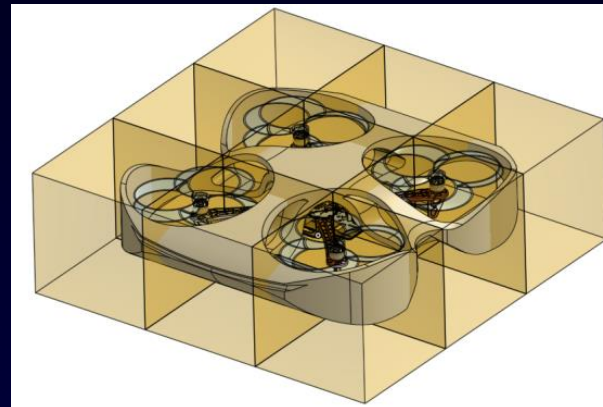
3. Our Comparative Enhancements



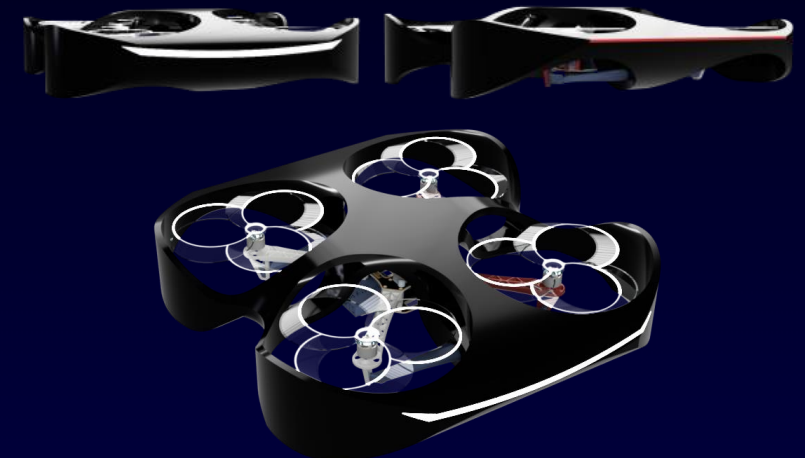
4. Software & PSOC Role



5. Hardware Process



6. Suggestions & Vision For The Future



1. What Our Project Does



Our project is a 3D-printed AI drone with an onboard camera for real-time computer vision and audio, enabling it for 3 modes:

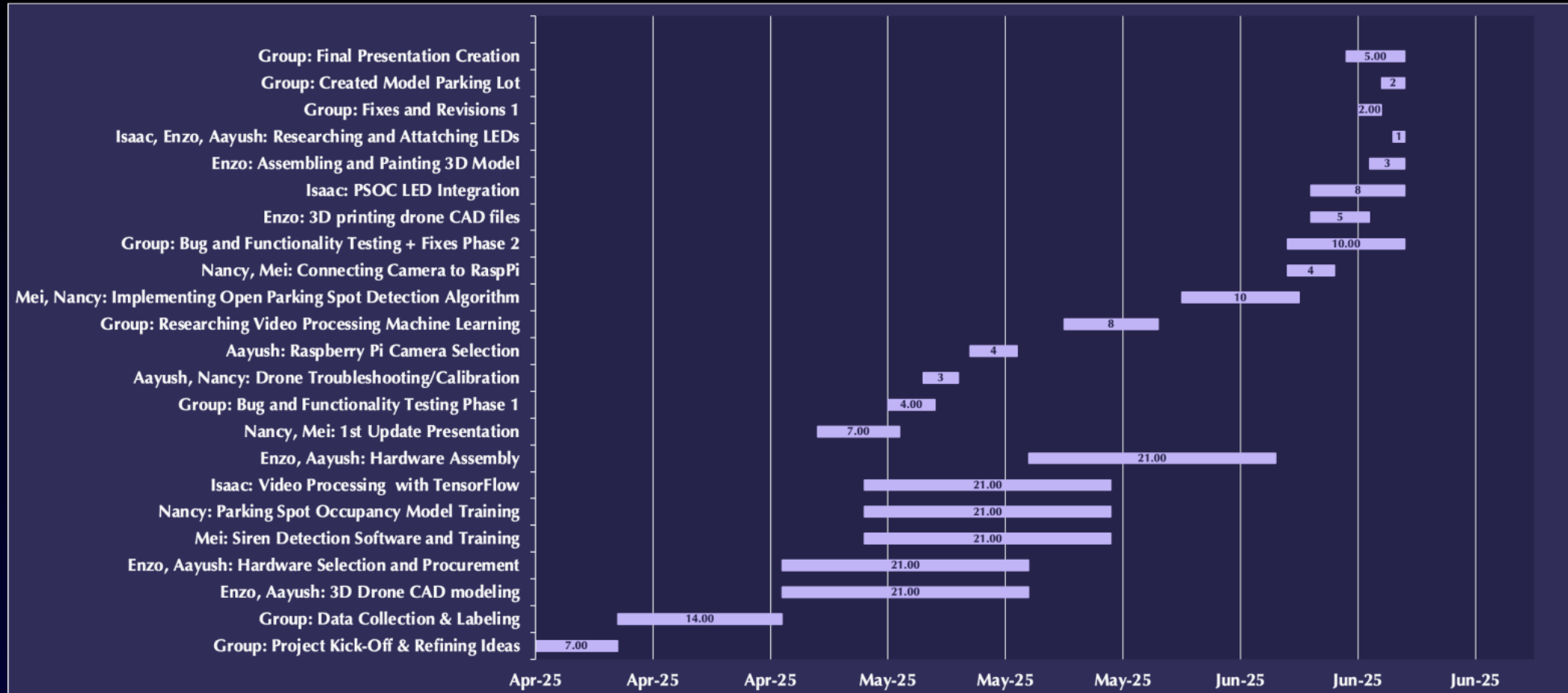
SCOUT: Identify vacant parking spaces

ALERT: Broadcast traffic-clearing alerts for ambulances

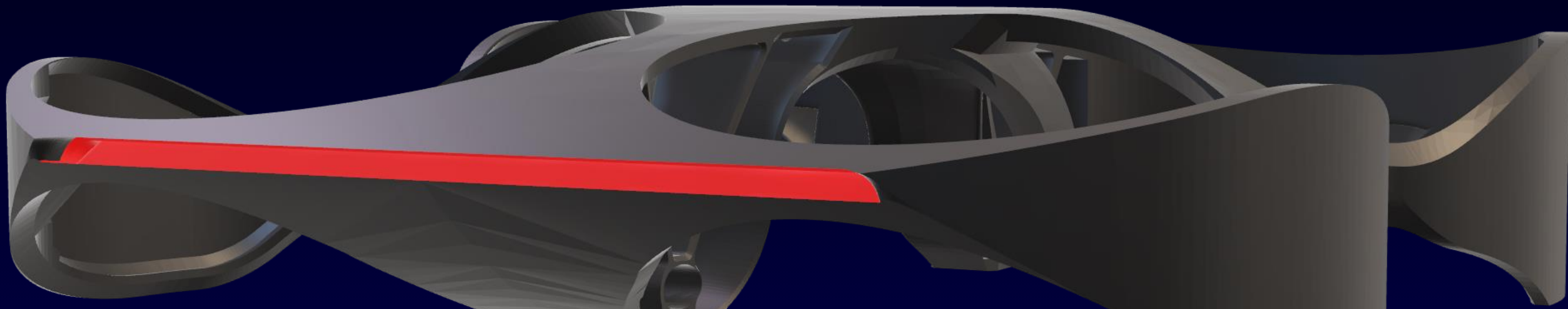
RUSH: Perform targeted vehicle tracking for law enforcement.



GANTT CHART



2. Why Is There A Need



2. Why Is There A Need

Law Enforcement

- Enable rapid, aerial tracking of stolen or suspect vehicles
- Provide real-time visual evidence and location data to support incident response.
- Introduces novel public direction methods

Parking Lot Routine

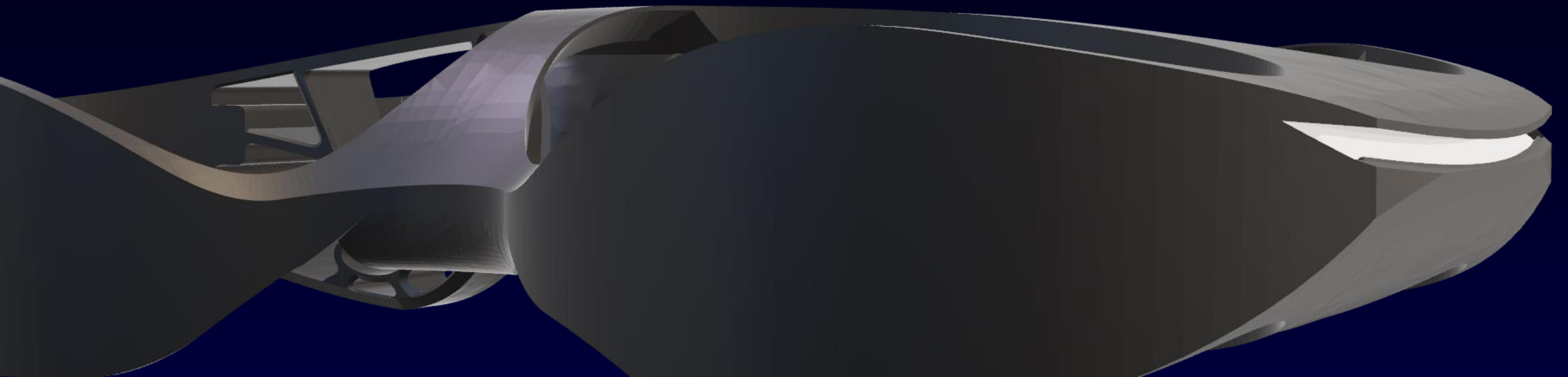
- 1 in 5 motor vehicle crashes in the U.S. occur in parking lots or garages
- Knowing the location of open parking slots beforehand will remove the need to multitask while in parking lots.

Civilian Lifestyle

- Lower- Eliminate frustration and time wasted searching for open spots by directing drivers instantly
- vehicle emissions and fuel consumption by minimizing “circle-the-lot” behavior
- Reduce parking-related stress and improve overall commute experience.



3. Our Comparative Enhancements



3. Our Comparative Enhancements



Maneuverability

Better flight control and agility.

Stronger motors to combat wind and remain stable for data collection

Portability

Design allows easy transport and deployment from the roof of a moving vehicle.

Made using lightweight material.

Versatility

Can be used for any kind of parking lot as it has a calibrating function that draws a perimeter.

Easy Integration

Involves a system that can easily be integrated to all cars.

Just need a flat surface to takeoff from and land on.

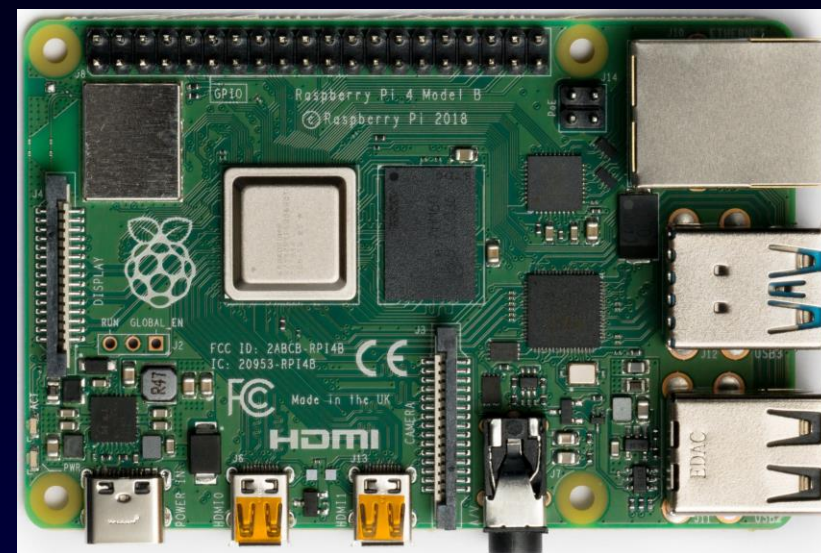
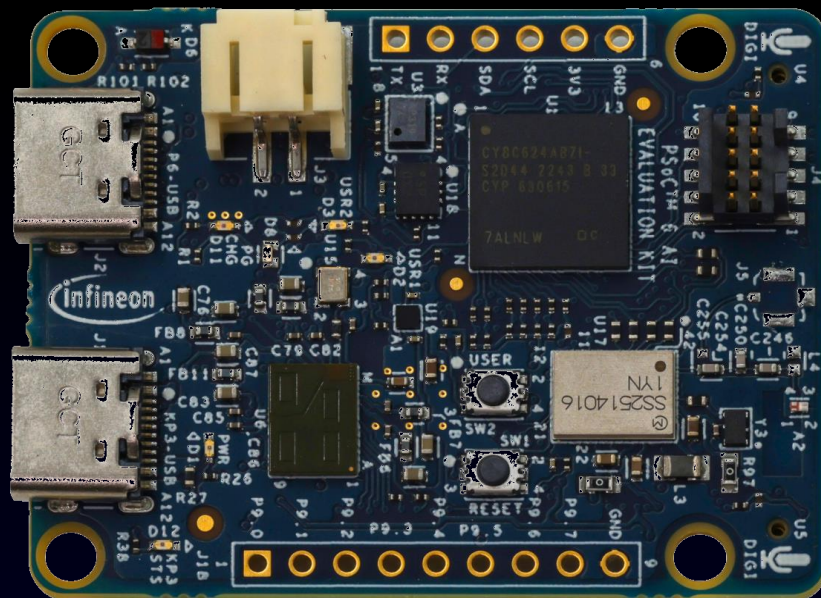
The GPS location of the empty spot is sent directly to your device
-> easy to navigate

Expandability

All components are individually found and put together, leaving a plenty of room for future upgrades and modifications.

Ma

4. Software



```
c:\Users\Han\Car-Parking-Slot-Occupancy-detection-model
Ultralytics YOLOv8.0.2 Python-3.10.11 torch-2.0.1+cpu CPU
Fusing layers...
Model summary: 168 layers, 3006038 parameters, 0 gradients, 8.1 GFLOPs

val: Scanning C:\Users\Han\Car-Parking-Slot-Occupancy-detection-model\Real-time-parking-lot-vehicle-detection-1\valid\labels...: 0% | 0/215 [00:00<?, ?it/s]
val: Scanning C:\Users\Han\Car-Parking-Slot-Occupancy-detection-model\Real-time-parking-lot-vehicle-detection-1\valid\labels... 1 images, 0 backgrounds, 0 corrupt: 0% | 1/215 [00:06<23:22, 6.55s/it]
val: Scanning C:\Users\Han\Car-Parking-Slot-Occupancy-detection-model\Real-time-parking-lot-vehicle-detection-1\valid\labels... 69 images, 0 backgrounds, 0 corrupt: 32% | 69/215 [00:06<00:09, 14.66it/s]
val: Scanning C:\Users\Han\Car-Parking-Slot-Occupancy-detection-model\Real-time-parking-lot-vehicle-detection-1\valid\labels... 125 images, 0 backgrounds, 0 corrupt: 58% | 125/215 [00:06<00:02, 30.69it/s]
val: Scanning C:\Users\Han\Car-Parking-Slot-Occupancy-detection-model\Real-time-parking-lot-vehicle-detection-1\valid\labels... 173 images, 0 backgrounds, 0 corrupt: 80% | 173/215 [00:06<00:00, 48.81it/s]
val: Scanning C:\Users\Han\Car-Parking-Slot-Occupancy-detection-model\Real-time-parking-lot-vehicle-detection-1\valid\labels... 215 images, 0 backgrounds, 0 corrupt: 100% | 215/215 [00:06<00:00, 30.98it/s]
```

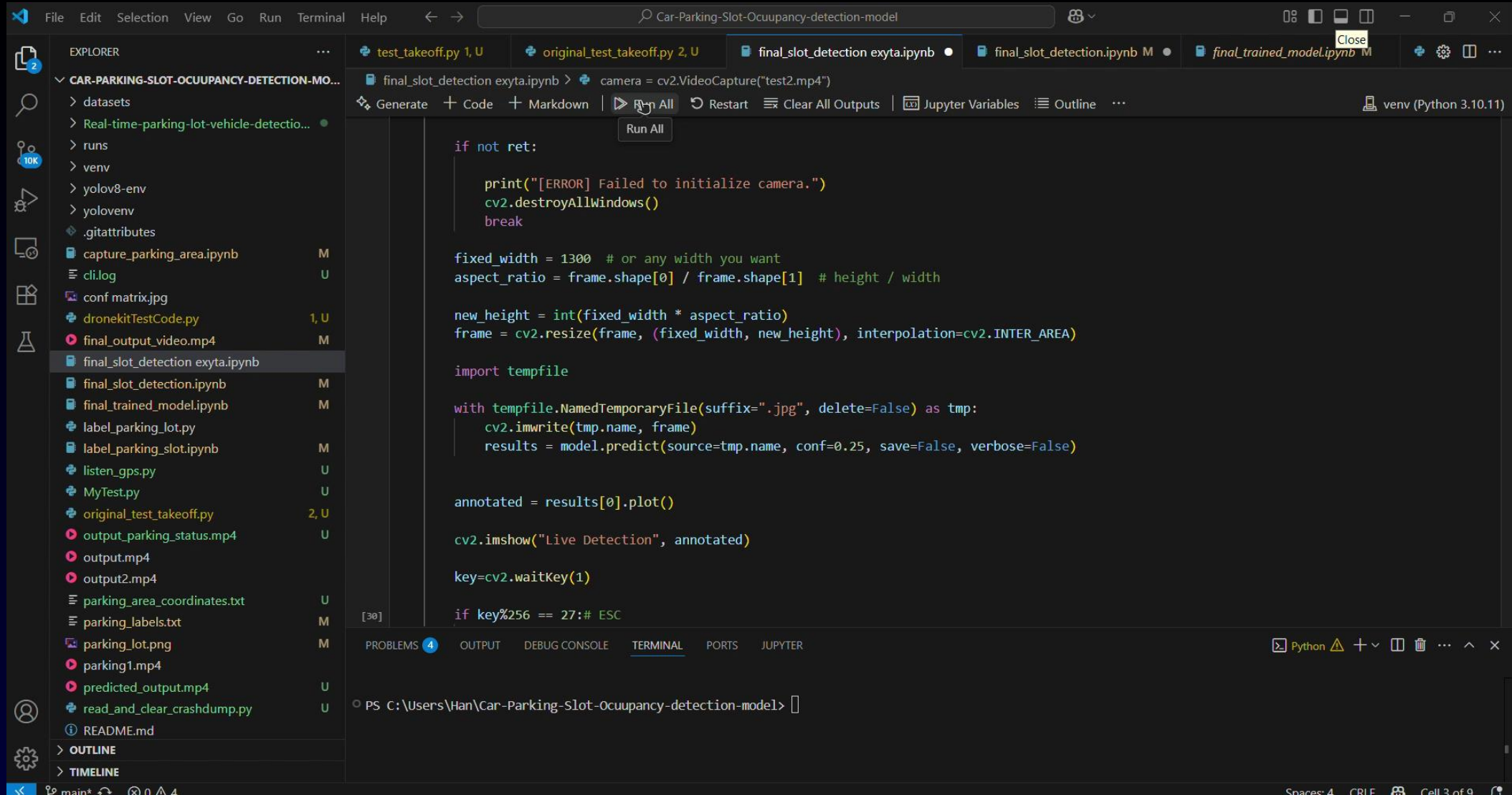
4. Software

Siren Detection Model

- Followed the instruction on Imagimob.com to build the siren detection model
- Connected to only detect after client has been connected

```
/* Print triggered class and the triggered time since IMAI init.*/  
unsigned long t = tick1 - start_t;  
char timeString[9];  
get_time_from_millisec_audio(t, timeString);  
printf("%s %s\r\n",LABELS[1],timeString);  
printf("Sending siren alert");  
tcp_client_send_siren_alert();  
printf("siren alert sent");  
// ADD CODE TO COMMUNICATE WITH RP HERE
```

Sample Video



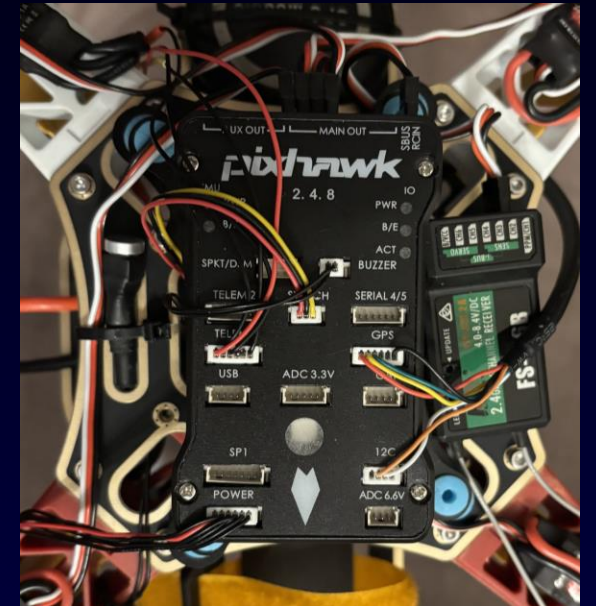
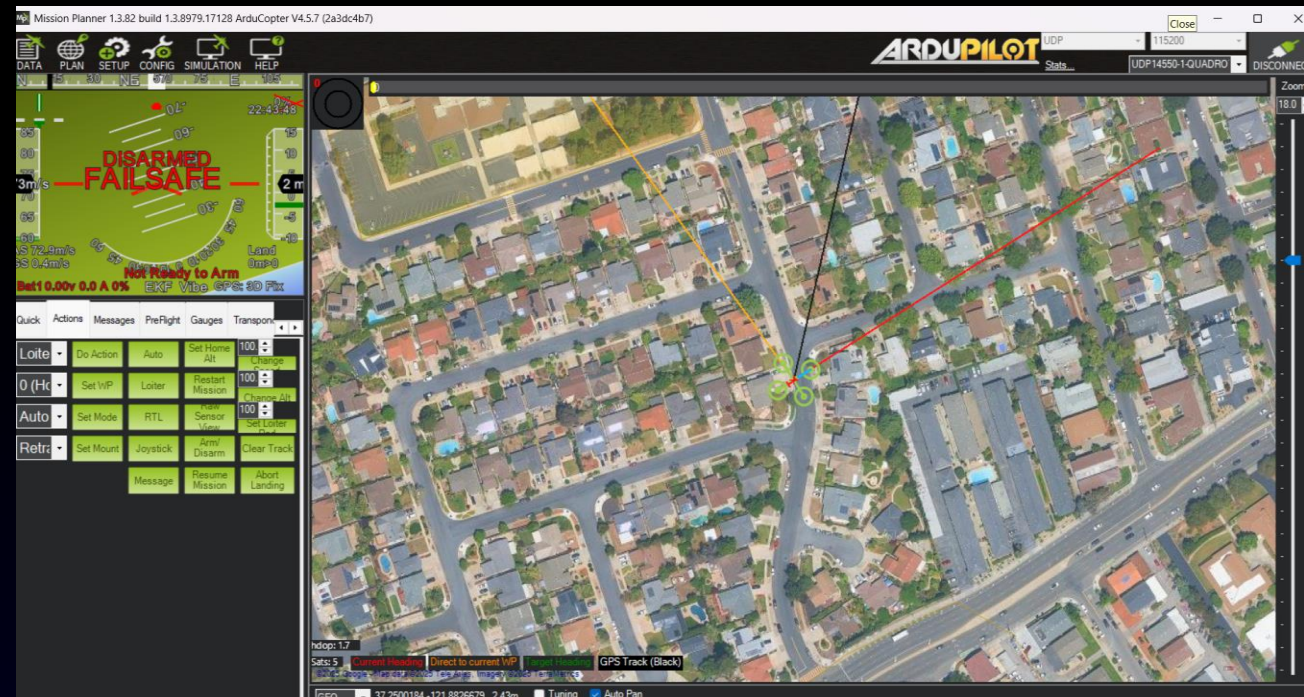
Drone Control

Hardware

- Pixhawk flight controller
- Raspberrypi 4B with Picamera2

Software

- ArduPilot ArduCopter firmware (runs on the Pixhawk)
- Mission Planner for setup and & telemetry
- Dronekit Python script for autonomy logic



DRONEKIT

Developer tools for drones

Algorithm

Calibration

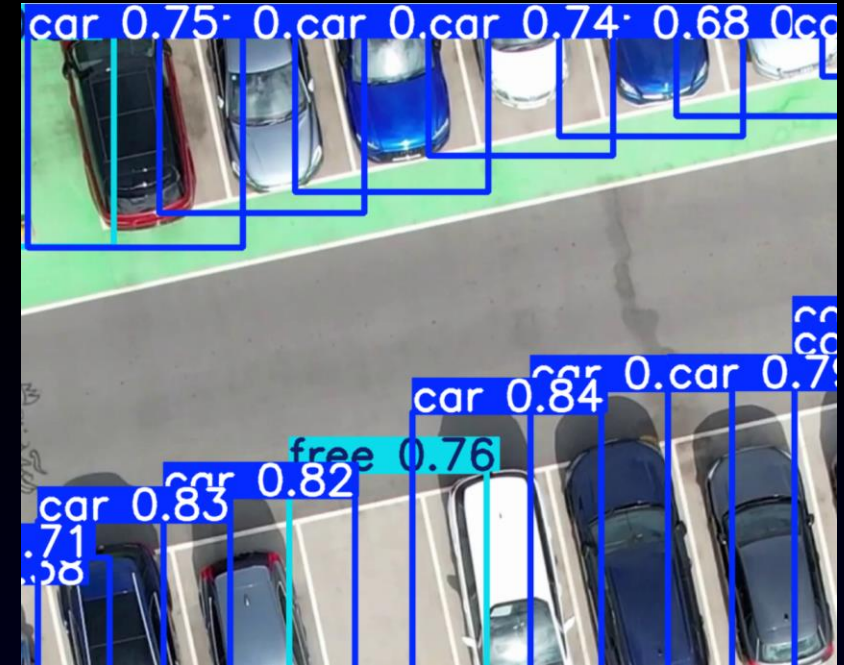
- Measure the perimeter of the parking lot
- Save the perimeter to ensure that the drone does not fly out of the area

Live Capture & Model Detection

- Capture a frame from the camera and run the detection model
- Determine the availability of an empty spot in the captured frame

Drone Maneuver

- If empty spot detected, move towards the spot
- If no empty spot found, generate a random coordinate within the parameter and move
- If centered above the empty spot, send the location to the device and land back on the takeoff spot



Code Snippets

```
results = detect_slots(frame)          #results = object that contains info about the one frame

for result in results:
    x_min = None
    y_min = None
    min_dist = float('inf')

    print(f"Number of boxes: {len(result.boxes)}")    #len(results.boxes) = number of boxes detected in frame

    for box in result.boxes:
        box_coordinates = box.xyxy[0].cpu().numpy()
        print("!!! Received Box Coordinates")
        cls_id = int(box.cls[0])
        label = result.names[cls_id]
        if label == "free":
            print("!!! Free Label Found")
            no_free_box_count = 0
            x = (box_coordinates[0] + box_coordinates[2]) / 2
            y = (box_coordinates[1] + box_coordinates[3]) / 2

            x_diff = x - frame.shape[1] / 2
            y_diff = y - frame.shape[0] / 2
            distance = math.sqrt(x_diff**2 + y_diff**2)

            if distance < min_dist:
                x_min = x
                y_min = y
                min_dist = distance
```

```
# Only send velocity if a valid free spot was found
if x_min is not None and y_min is not None:
    err_x = frame.shape[1] / 2 - x_min
    err_y = frame.shape[0] / 2 - y_min
    (variable) err_y: Any

    draw_position_grid(err_x, err_y)

    if(abs(err_x) <= centered_x_threshold and abs(err_y) <= centered_y_threshold):
        print("The drone is CENTERED")
        vehicle.simple_goto(vehicle.location.global_relative_frame) # hover in place
        # send the location to the device
        loc = vehicle.location.global_relative_frame
        msg = f"{loc.lat:.8f},{loc.lon:.8f},{loc.alt:.2f}"
        udp_sock.sendto(msg.encode("utf-8"), (laptop_IP, laptop_port))
        print(f"✈️ Sent location to laptop: {msg}")
        #Go back home and land
        vehicle.simple_goto(home_location)
        #hover for a few seconds
        time.sleep(5)
        print("Landing...", flush=True)
        vehicle.mode = VehicleMode("LAND")

        #Close vehicle object before exiting script
        time.sleep(5)
        vehicle.close()
        print("Done", flush=True)

    else:
        print("-----")
        vx = err_x * 0.0001 # Adjusted gain
        vy = -err_y * 0.0001

        if vx > 0:
            print("➔ Drone move RIGHT (positive x error)")
        elif vx < 0:
            print("➔ Drone move LEFT (negative x error)")

        if vy > 0:
            print("⬆ Drone move FORWARD/NORTH (positive y error)")
        elif vy < 0:
            print("⬆ Drone move BACKWARD/SOUTH (negative y error)")
        print("Centering Velocity Sent")
        send_ned_velocity(vy, vx, 0, 0.0001)

    else:
        #it doesn't detect any free box in the frame
        if (no_free_box_count < 5):
            no_free_box_count += 1
            print(f"No Free Box Count = {no_free_box_count}")
            break

        print("No free spots. Generating new random coordinate")
        target_location = random_coordinates_within_perimeter(home_location, perimeter)
        fly_to_if_unique_and_check_boxes(vehicle, home_location, target_location.lat, target_location.lon, perimeter)
        break
```

Challenges

Raspberry pi

- Unable to capture video from the pi camera without starving the MAVLink GPS code of CPU time, which caused the GPS to glitch
- Difficulty of running Mission Planner on the computer while running the detection model simultaneously on the raspberry pi

PSOC 6 AI Board

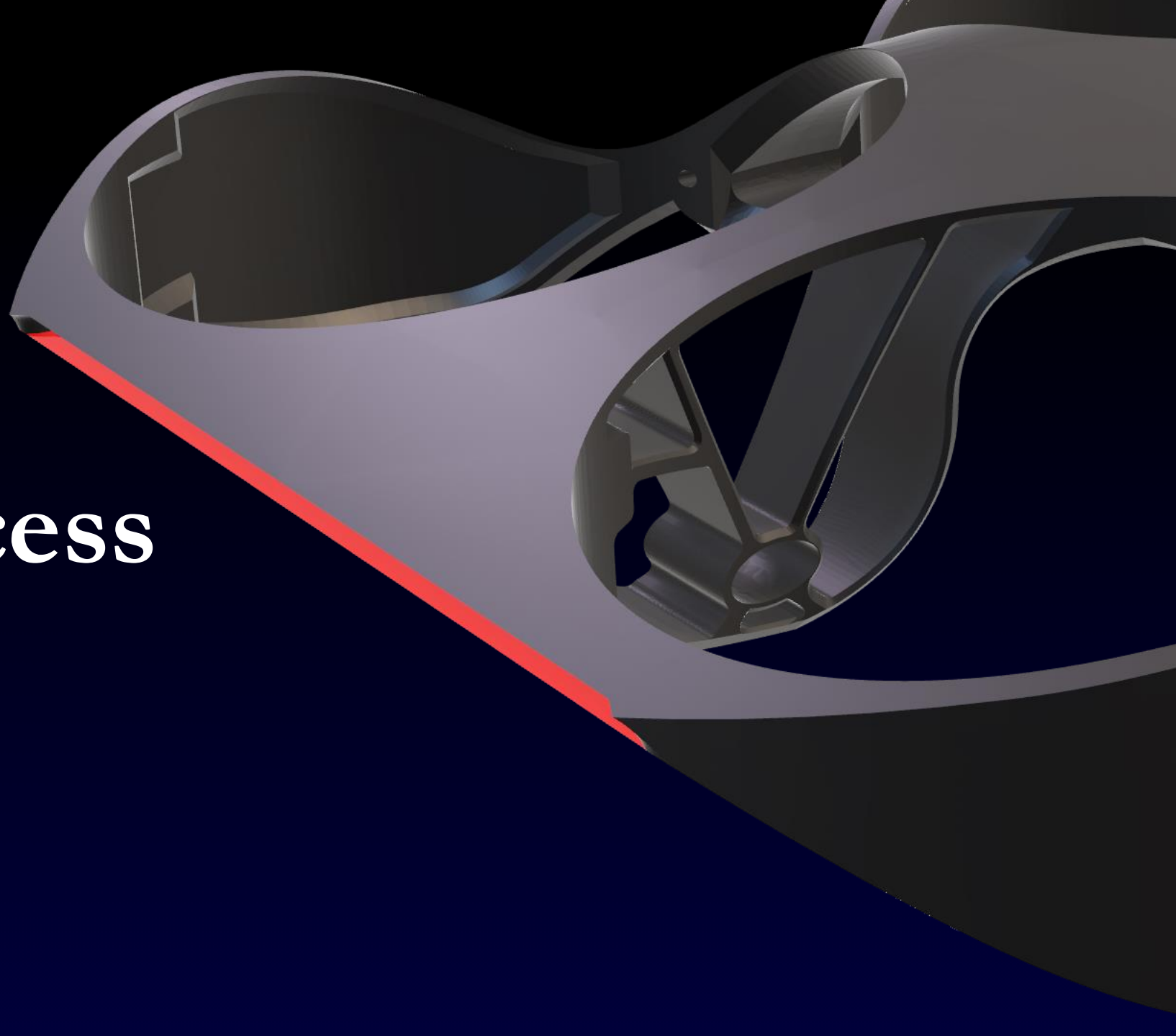
- Difficult time integrating multiple tasks at once.
- Could not run two tasks in parallel, commands like `vTaskStartScheduler()` did not work as intended.
- Even when priority was set to be the same, audio task would always take priority if put in the same block of code.
- Could not connect using Bluetooth and HTTPS, had to connect using TCP server instead with minimal security
- Many warnings during compilation even though program builds and runs successfully, very hard to guarantee memory handling is working as intended

Challenges cont.

Hardware

- Our first drone had an issue taking off – the battery caught on fire
- During test flights, the drone landed sideways numerous times, causing the propellers to break off
- Calibration
- Connectivity issues
- GPS consistency
- ESC wiring causing movement difficulties
- Lack of options for connecting the PSOC to Pixhawk

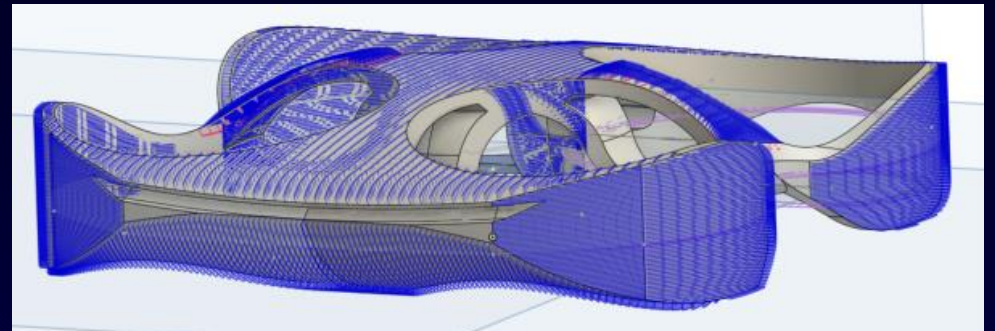
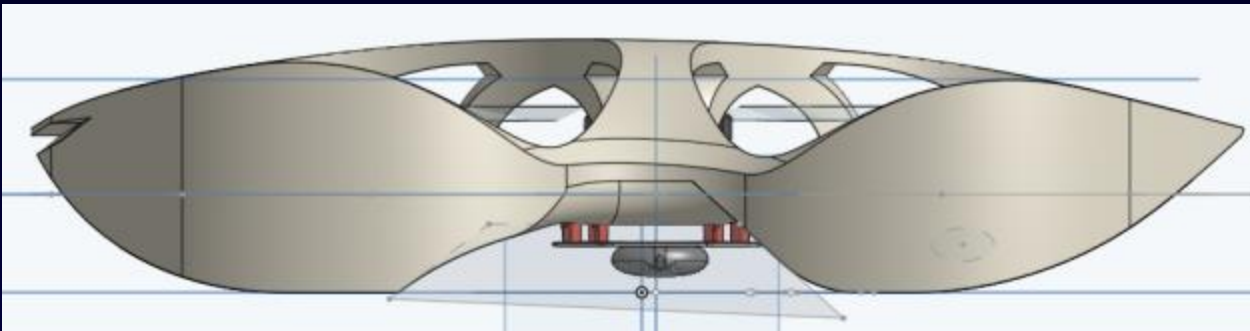
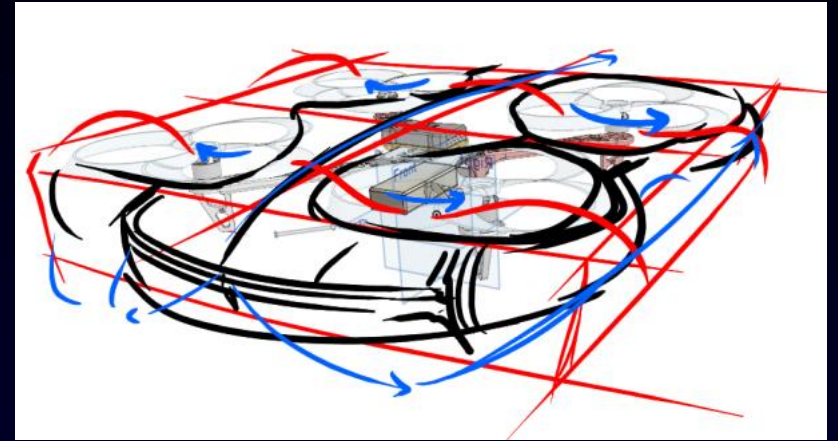
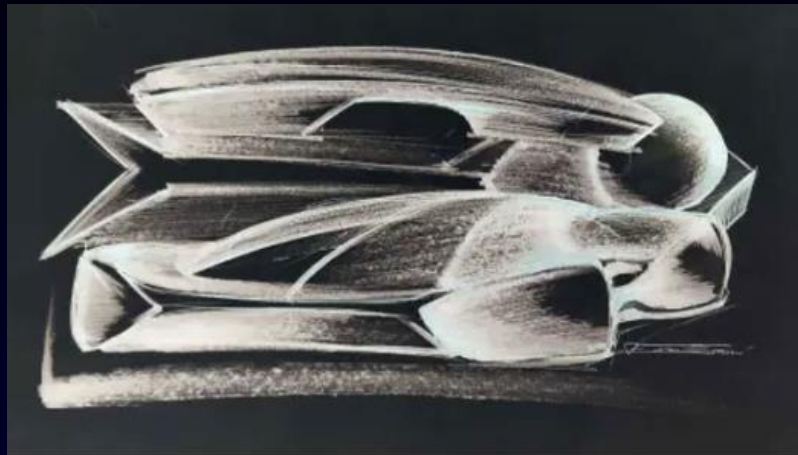
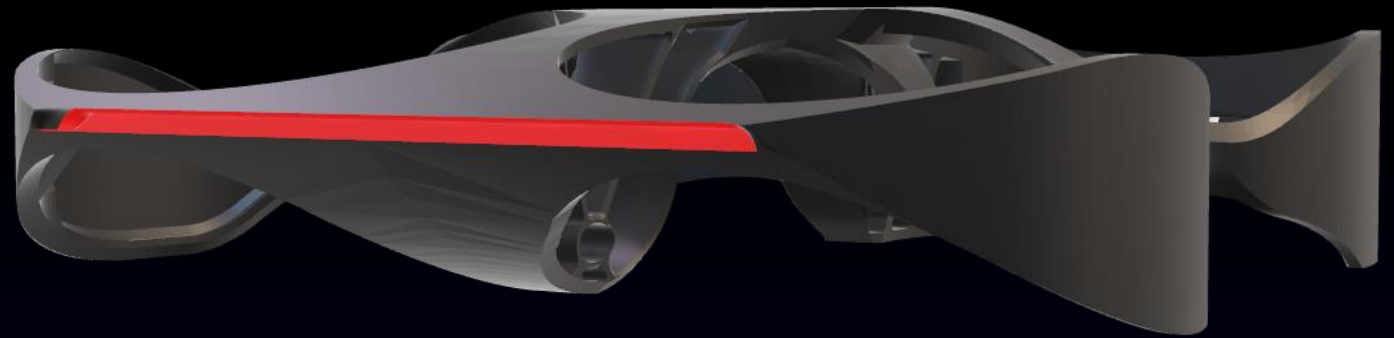
5. The Process



5. The Process

DESIGN

Sharp / Sleek / Quiet
Aerodynamic Surfacing
Lexus Style
Lucid Slit Lights
Strong Core
Designed Entirely from
scratch with Onshape



5. The Process

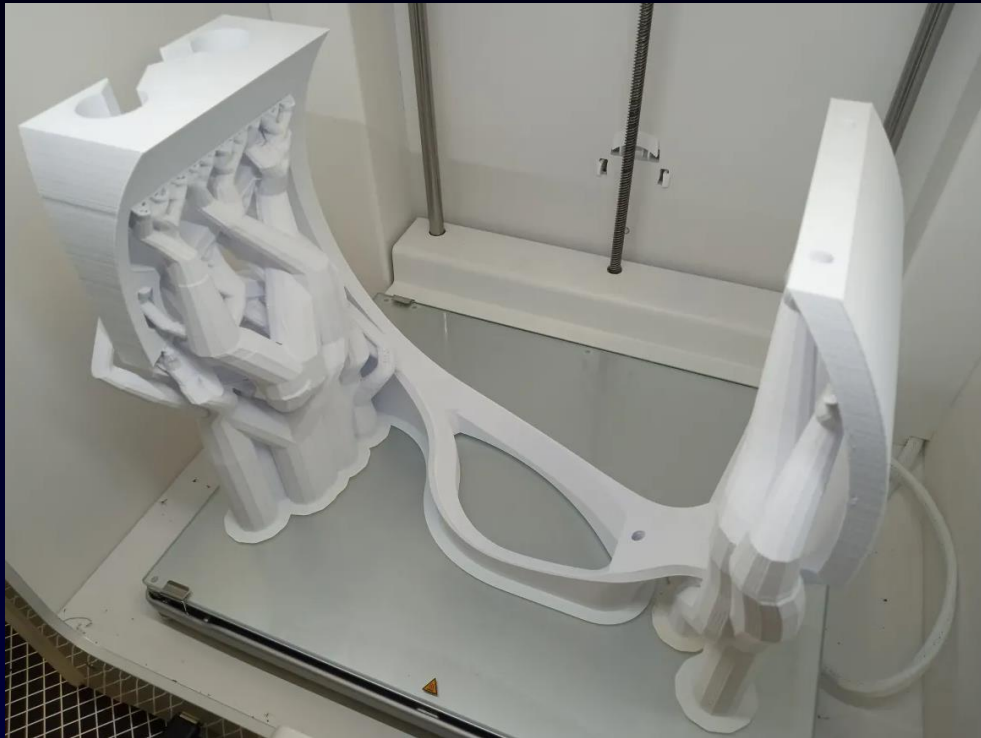
CONCEPT



5. The Process

MANUFACTURING

Printed with PLA+ on Ultimakers
with the support of DE ANZA DMT
machinery.

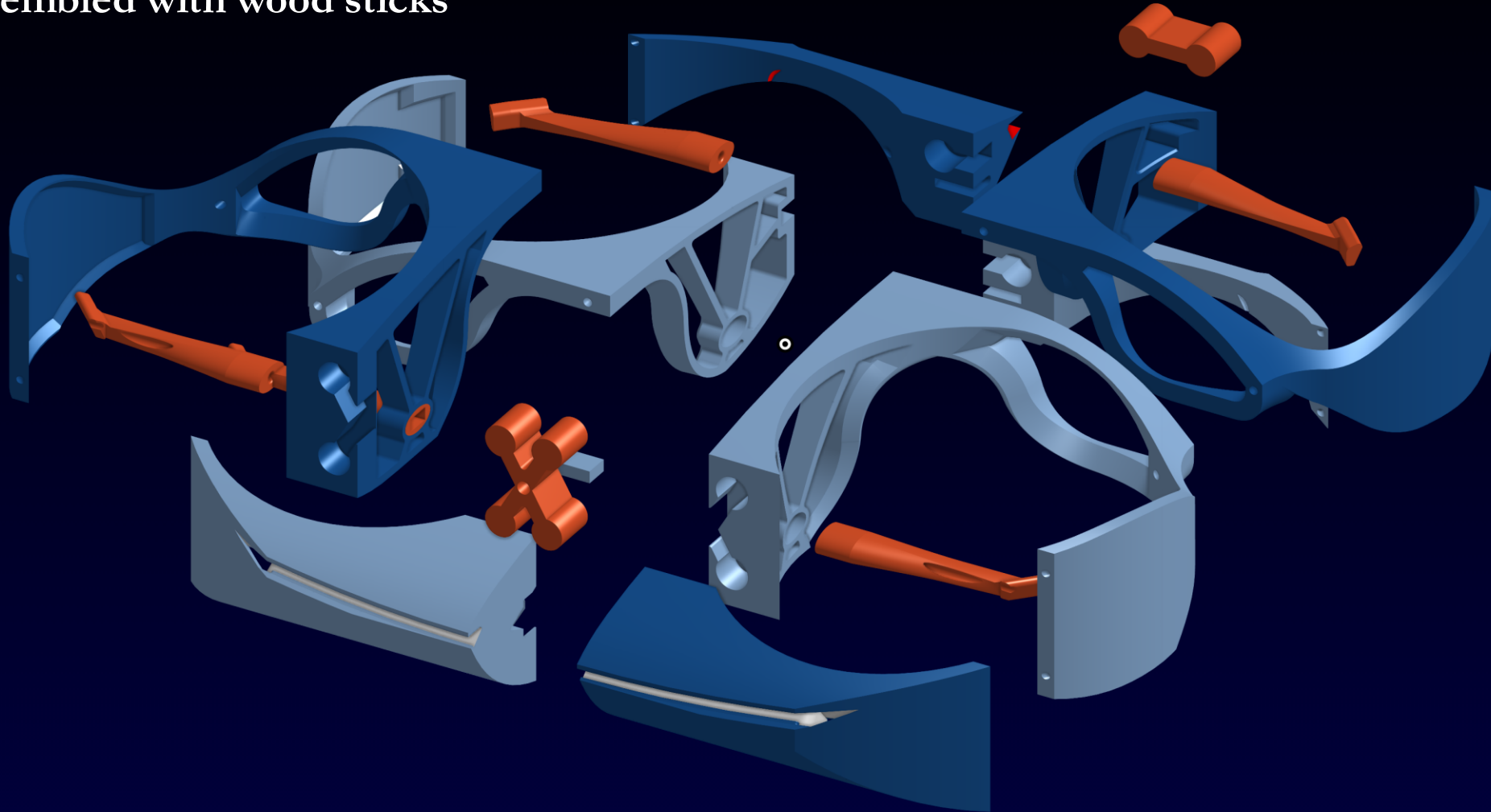


5. The Process

ASSEMBLY



Outer Body Designed Out of 8
Pieces, assembled with wood sticks



5. The Process FINISHING



Sanding + Priming + Car Paint + Polish



6. Suggestions & Vision For The Future



6. Our Experience with the PSOC

Strengths

- Built-in input sensors for easy and fast data collection, as well as great compatibility with Deepcraft enabling fast program development
- Various built-in models that make the development of AI
- Easy to program into the board using ModusToolbox
- Multiple sensors that allow for a wide range of projects
- The board is compact and portable

Suggestions for Improvement

- Lack of support system and examples make it difficult to resolve problems
- It is a complex process to combine multiple models together
- Poor memory management
- Optimization: ready-to-deploy models do work, but need more data for accuracy
- Inconsistency in the input sensors - some sensors seem to work better than the others

6. Our Vision



Detect and Fly

- We will make the raspberry pi be able to handle autonomous flight while capturing video and running the model detection
- Improve the autonomous flight program for more reliability and precise control

Improved Utilization of PSOC

- Further use of the PSOC data collection and processing capability to assist the autonomous flight
- Explore more sensors and achieve combination of multiple models

More Modes

- Achieve the "Rush" mode, which tracks a vehicle
- Enhance the "Alert" mode to have the PSOC do more than triggering the LED